

深入理解 AI Agent：設計原理與工程實踐

習題參考答案 思考題參考答案

李博杰 著 · 繁體中文版

大字寬鬆排版 正文 14pt／行高 1.9

本章目錄

1. 第一章 AI Agent 入門
2. 第二章 上下文工程
3. 第三章 使用者記憶和知識庫
4. 第四章 工具
5. 第五章 Coding Agent 與程式碼生成
6. 第六章 Agent 的評估
7. 第七章 模型後訓練
8. 第八章 Agent 的持續進化
9. 第九章 多模態與即時互動
10. 第十章 多 Agent 協作

本檔案彙總全書十章思考題的參考答案提綱。思考題多為開放性問題，答案不唯一。參考答案為 AI 生成，人工略作審校，僅供讀者對照啟發之用。建議讀者使用 LLM 結合書稿內容進一步討論這些問題。

第一章 AI Agent 入門

1. (★★) 如果你只能給一個 Agent 系統增加一項能力——更強的模型、更豐富的上下文、還是更多的工具——你會選哪個？在什麼條件下你的選擇會改變？

對應「大腦/眼睛/手腳」公式，先找短板：通常優先補上下文，即補充觀察空間（observation space）。若任務超出模型推理能力，換更強模型。若動作空間不足（例如無法存取公司內部系統），加工具。判斷依據是分析失敗軌跡，定位瓶頸在感知、決策還是行動。

2. (★★★) ReAct 迴圈中，Agent 的每一次 LLM 呼叫都會看到完整的歷史軌跡。隨著軌跡增長，這種設計的成本是二次方增長的。有沒有辦法在不丟失關鍵資訊的前提下打破這個二次方？

可行手段：上下文壓縮——摘要早期軌跡、只保留結論與關鍵狀態（第二章的多層壓縮）；外部化學習——把中間結果寫入檔案/知識庫，按需檢索而非常駐上下文；拆分為子 Agent。

3. (★★) 「模型即 Agent」範式意味著模型在工具呼叫決策上越來越自主。但本章論證了 Harness 工程的重要性反而在增加。這兩個趨勢如何共存？Agent 框架未來的核心價值體現在哪些方面？

馬與韁繩隱喻：模型越強、自主空間越大，出錯影響面越大，越需約束、驗證、糾正。框架價值從「編排 LLM 呼叫」轉向 Harness 五要素中的保障層：權限分類、熔斷器、錯誤恢復、上下文壓縮、工具生態。

4. (★★) 消融實驗中「工具結果回饋」的缺失導致 Agent 陷入無限迴圈。在生產環境中，除了工具結果缺失，還有哪些情況可能導致 Agent 無限迴圈？你會設計怎樣的偵測和終止機制？

其他誘因：工具反覆報同一錯誤、幻覺呼叫不存在的工具、上下文被壓縮丟失關鍵狀態、思考過程被剝離導致模型 API 報錯、任務本身無解。機制：設最大迭代次數等停止條件；偵測重複呼叫（相同工具+引數指紋）；超過失敗閾值升級人工干預。

5. (★) 本章用感知、行動、策略三個維度分析了五個 Agent 產品。請選擇一個你日常使用的 AI 產品，用這三個維度進行分析，並思考它的架構設計是否合理。如果由你來設計這個 AI 產品，有哪些改進空間？

開放題。要點：仿照章中表格，寫出眼睛（能看到什麼資訊源）、手腳（動作空間是否開放式、能否內部思考）、策略（Agent 執行迴圈的模式）。

6. (★★) 如果你要設計一個專門處理航班訂票的客服系統，你會選擇 workflow 模式還是自主 Agent 模式？有沒有可能在同一個系統中混合使用兩種模式？

主體用 workflow：身份核實→搜尋→付款→預訂四節點，保證「付款前不能預訂」等合規順序，且把提示注入攻擊面限制在單節點內。開放性環節（理解需求、改簽、航班取消推薦替代方案）切換自主 Agent。高風險操作（大額付款、退款）加人工確認。

7. (★★★) 護欄部分提到了工具風險評級。如果一個工具在大多數情況下是低風險的，但在特定引數組合下變為高風險（如 `delete_file` 刪除普通檔案 vs 刪除系統檔案），你會如何設計動態風險評估？

評級物件從「工具」細化到「工具+引數」：按可逆性、權限、影響面在呼叫時計算風險。用基於規則的確定性檢查（路徑黑白名單、正則）而非模型判斷。驗證應只看結構化資料，防提示注入操縱。

8. (★★) 本章的 Agent 產品表格中，所有 Agent 的動作空間都是「開放式」的。一個受限的動作空間（比如只能從預定義選項中選擇）在什麼場景下反而優於開放式？

高合規、高風險、錯誤不可逆場景：如退款、付款，受限選項即「約束」，天然防呆，從設計上讓錯誤無法發生。

9. (★★) 人工干預機制要求 Agent 能「優雅地移交控制」。但在實踐中，使用者可能不線上、響應很慢、或者給出模糊的指令。此時 Agent 應該怎麼辦？

Fail-safe：高風險操作在無確認時暫停而非預設執行；先做可逆的低風險部分，文件記錄高風險部分，便於人類決策、Agent 恢復；用非同步溝通工具（訊息、郵件）通知並設超時策略；指令模糊時用意圖澄清。

10. (★★★) 引言指出「好的設計原則應該穿越模型的迭代週期」。試舉一個你認為可能會隨模型進步而過時的當前 Agent 設計原則，並說明理由。

示例：少樣本提示、精細提示工程——模型零樣本泛化增強後收益遞減；透過限制取樣實現的嚴格工具呼叫格式——目前 SOTA 模型工具呼叫格式已經比較穩定；人工編排的多步工作流——模型指令遵循能力提高後不再需要。

第二章 上下文工程

1. (★★★) 實驗 2-3 發現，滑動視窗對話歷史會導致 Agent 反覆執行相同的工具呼叫。但完整保留歷史又會讓上下文不斷膨脹。設計一種策略，既能避免資訊丟失，又能控制上下文長度，且不破壞 KV Cache 字首。

①用壓縮代替丟棄：訊息只追加不刪改，接近閾值（如視窗 80%）時批次壓縮舊 tool results。②分層機制：大輸出落盤留摘要、噪聲直刪、歸檔式摘要保留脈絡。③子 Agent 隔離，讓中間狀態不進主上下文。

2. (★★) Qwen3 的 Chat Template 思維鏈保留機制只保留「最後一個真實使用者訊息之後」的思考。如果一個 ReAct 迴圈跨越了上百輪工具呼叫，累積的思考內容可能消耗大量上下文。你會如何修改這個機制來應對超長迴圈？DeepSeek R1 曾要求剝離全部歷史思考，而 DeepSeek V4 反轉為強制回傳全部 `reasoning_content` ——對比這兩種相反的策略，各有什麼利弊？這個反轉說明了什麼？

修改方向：滑窗保留——完整保留最近若干輪思考，視窗外按 token 預算（而非固定輪數）觸發滾動壓縮，產出結構化狀態列（當前目標、已確認事實、已排除路徑、待辦），壓縮只發生一次且位置固定，快取重建代價是一次性而非每輪付出。R1 剝離：省 token、字首穩定快取友好，且與訓練分佈一致（歷史 CoT 從不在輸入中）；但每輪從零推理，長程計畫丟失、易重複犯錯。V4 強制回傳：思路連貫、長程 agent 任務表現更好；但 token 成本高、每輪字首膨脹，且無法從非 think 模式無縫切換。反轉說明：對純對話場景思考是廢料，對 agentic 場景思考是狀態——行業實踐已倒向後者。

3. (★★) 上下文感知壓縮實驗中，從約 148K 個字元壓縮到約 2,000 個字元，這種極端的壓縮是否存在「不可逆資訊損失」的風險？如何解決？

有風險，壓縮是有損投影，問題落到未保留的維度就壞了。解法：「有損壓縮＋無損索引」，每條事實帶來源 URL 可回溯；原始輸出存磁碟、只看摘要預覽；顯式保留優先順序——架構決策、語義完整性（時間、公司名）、驗證狀態、UUID/hash 等識別符號原樣保留；適應性視窗化推遲壓縮時機。

4. (★★) Agent 狀態列將隱式狀態顯式化。但如果狀態列本身包含了錯誤資訊（比如工具計數器出了 bug），Agent 可能基於錯誤的資訊做出有害的決策。這種「元資訊可靠性」問題如何緩解？

模型幾乎無條件相信狀態列，錯誤會原樣傳導。緩解：①用確定性程式碼維護，絕不讓 LLM 批次統計長歷史（要用也逐條抽取、程式碼彙總）；②把狀態列準確率當一線生產指標盯；③資訊只來自對真實世界的可靠觀測，防狀態列投毒。

5. (★★) 提示工程消融實驗表明，資訊組織的混亂導致成功率下降 30% 以上。但在實際開發中，系統提示詞往往由多人在不同時間維護。你會用什麼工程實踐來防止系統提示詞的「熵增」？

①把提示詞當程式碼：版本控制、評審，產品經理定業務規則、工程師負責編碼；②用 Tau-Bench 類基準測試做迴歸測試，改動前後跑消融實驗定位影響；③強制結構化：SOP 流程驅動而非規則堆砌，XML/Markdown 分層；④片段按「可快取/破壞快取」分類命名，動態內容歸到快取邊界後；⑤膨脹內容拆成 Skills 按需載入。

6. (★★★) 本章提出「上下文學習本質上是檢索而非推理」。如果這個論斷成立，當前所有基於「把更多資訊塞進上下文」的最佳化方向都需要重新審視。你認為應該如何突破這一侷限？

給「只有一半的檢索引擎」補提煉層：①上下文蒸餾/狀態列，用程式碼提前算好結論供直接檢索；②主動壓縮，把原始記錄換成高密度結構化知識；③子 Agent 隔離，噪聲不進主上下文；④互動作為第三軸，外部儀器觀測寫回模型想不出的新資訊；⑤前沿方向：可編輯、可組合的 KV Cache 「筆記」，及跨會話記憶沉澱。

7. (★★★) Skills 的漸進式披露只在 Agent 判斷需要時才載入完整內容。但這個判斷本身依賴模型的能力——如果模型不知道自己不知道什麼，就無法正確觸發 Skill 的載入。這個「元認知」問題如何解決？

①Skill 的元資料（名字、描述）常駐上下文，讓模型始終「知道自己擁有什麼」；②Skill 的 description 寫成路由條件而非功能介紹：「Use when / Don't use when」，避免寬泛描述。

8. (★★) Skills 機制中，Agent 從 SKILL 檔案中動態讀取提示詞之後，後續的操作能否正確遵從這些指令？不同的模型對 Skills 模式的支援有什麼區別？

取決於 Skill 的注入方式：注入 system prompt 遵循最強但破壞 KV Cache；作為普通檔案讀到上下文中間，模型的指令遵循可能較差；注入到上下文末尾，指令遵循較好，但每次工具呼叫都需要重新計算 skill 部分的 KV，成本較高。

9. (★★★) 本章強調動態資訊（如系統時間戳、工具列表順序）的變化會破壞 KV Cache 字首命中。在一個擁有大量工具且工具集頻繁變動的生產系統中，你會如何設計上下文佈局來最大化快取命中率？

①少量穩定核心工具（如七個）＋通用執行器，具體能力走 Skills 漸進披露，工具定義凍結在靜態字首、固定順序；②子 Agent 與父 Agent 字首保持相同。

第三章 使用者記憶和知識庫

1. (★★) 在使用者記憶系統中，當同一使用者在不同會話中提供了矛盾資訊（比如兩次提到不同的家庭住址），記憶系統應該如何處理這種衝突？

用 Mem0 式「提取—對比—決策」流水線：先向量檢索出相近舊記憶，再由 LLM 判定 ADD/UPDATE/DELETE/NOOP，如「搬到上海」應 UPDATE 覆蓋「住在北京」；版本化：地址類資訊只保留最新版並標記時間戳，工作經歷類保留完整歷史；檢索側可借上下文字首（人物、時間、意圖，如電匯三次修改案例）判斷哪條最終有效。

2. (★★) 上下文感知檢索將原始文件的上下文附加到每個分塊。但如果原始文件本身結構混亂或存在矛盾資訊，這種方法可能傳播甚至放大錯誤。你會如何在檢索階段引入「資訊質量」訊號？

借鑑「知識庫時效與治理」：給分塊附加版本號碼、生效/失效時間、來源等元資料，檢索時過濾已失效內容，或在字首中顯式標註「此條已於某日廢止」；重排序階段把來源權威性、時間新鮮度納入打分，而非只看語義相關性；索引期讓生成字首的 LLM 順帶偵測塊間矛盾並標記，類似記憶的版本化衝突偵測。

3. (★★★) 智慧體化 RAG 讓 Agent 主動決定何時搜尋、搜尋什麼、以及是否需要繼續搜尋。但如果模型不知道自己不知道什麼，就無法正確觸發搜尋。這個「元認知」問題如何解決？

①在 prompt/skills 中把「評估資訊是否充分」固化為顯式步驟：如實驗 3-9 中先並行檢索子問題，發現缺「前科如何影響過失罪量刑」這一關聯，再二次檢索；②讓輕量元資訊常駐上下文提供全域性視野：例如 JSON Cards 概覽、OpenViking 的 L0/L1 摘要，使 Agent 知道「庫裡有什麼」。

4. (★★) 多模態資訊提取將圖表轉為文字描述後再進行檢索。這個「翻譯」過程可能丟失視覺資訊中的空間關係。舉一個具體例子，說明純文字描述無法完整傳達的圖表資訊，並設計一種保留該資訊的方案。

例：系統架構圖中的邏輯關係，折線圖中兩條曲線的交叉點位置，或 PDF 表格中單元格與表頭的行列對應。方案一：原生多模態處理；方案二：提供多模態圖片分析工具。

5. (★★★) Rich Sutton 的「苦澀的教訓」認為通用方法（搜尋和學習）最終會勝過手工設計的特徵。本章建構的整個知識系統（分塊策略、索引結構、檢索管道）是否本身就是一種「手工設計」？如果模型能力足夠強，這些設計是否會被簡單的「全量輸入」所替代？

確實是手工設計，部分環節（分塊、融合調參）可能隨長上下文而弱化；但黑貓白貓案例表明「全量輸入」也不夠：注意力是軟檢索，跨文件聚合統計仍需索引期預提煉；知識過期更新、權限/租戶隔離、可審查性、成本這些工程約束與模型能力無關；且檢索與索引期 LLM 提煉本身就是「搜尋+學習」的通用方法，並非與苦澀教訓對立。

6. (★★★) 隨著模型能力的提升，你認為領域知識庫還重要嗎？未來強大的基座模型是否有可能包含領域知識庫中所有的資訊，從而不再需要領域知識庫？

仍重要：訓練資料有截止日期，知識庫可隨時更新；企業內部流程、私有判例等根本不在公開語料中；多使用者共享需權限過濾與租戶隔離，引數中的知識無法按呼叫者裁剪；外部儲存可審查、可版本控制、可下線失效內容，引數記憶難以做到；即使走引數化路線（後訓練 / User as Engram），也面臨「記住容易，能用來做多跳推理難」的難題。

7. (★) RAPTOR 透過由下而上的層次摘要建構樹形索引，GraphRAG 透過實體關係建構圖結構索引。這兩種結構化索引分別擅長回答什麼型別的查詢？

RAPTOR：從宏觀概念逐步鑽取細節的「跨層穿梭」式查詢，如先定位「SIMD 指令集」摘要再下鑽到 SSE 細節，兼顧總覽與細節兩種粒度。

GraphRAG：多跳關係推理（「我的醫生所在醫院的地址」沿關係鏈走訪）與實體消歧（兩個「張醫生」是不同節點）等「A 和 B 有什麼關係」類查詢，社群摘要還提供主題聚類。

8. (★★) 檔案系統範式將知識組織為類似檔案系統的層次結構。這種方式和傳統的向量資料庫 RAG 相比，在什麼場景下更有優勢？

純文字可被使用者直接閱讀、編輯、修正，可用 Git 版本控制與回滾，適合需要人機共同維護、審查知識的場景；Agent 有 write_file 能力即可自主記錄經驗，形成記憶自進化迴圈（外部化學習）；L0/L1/L2 漸進披露使多數查詢到 L1 即可決策，省 token；前提是像 Wikipedia 一樣建立交叉連結和索引頁，否則孤立檔案越多越難檢索。

9. (★★★) 從結構化資料（如司法判決資料庫）中自動發現「裁判因素」和「因素重要性層級」，本質上是讓 Agent 從資料中歸納規則。這種資料驅動的知識提取是否能達到人類專家手工編寫規則的質量？

優勢：如 CAIL2018 實驗，「自下而上」因子發現更貼合資料而非人類先驗，能捕捉散落在成千上萬判例中、專家難以顯式寫出的隱性權衡經驗，且可量化。侷限：LLM 提取出錯會造成知識汙染，資料本身的偏差會被繼承，聚類原型只反映相關性、說不清因果。折中：資料驅動建模＋專家稽核 Schema 與結果，模型驅動提問、統計支撐解釋。

第四章 工具

1. (★★) MCP 標準將工具定義從 Agent 框架中解耦了出來。但標準化也意味著複雜的工具互動模式（如流式輸出、雙向通訊、有狀態會話）可能難以在標準協定中表達。你認為 MCP 未來最需要擴充套件的能力是什麼？

最需要跨會話的事件驅動能力。MCP 主體是請求-回應式，notifications、progress、sampling、elicitation 等原語都限於保持連線的單個會話內，通知只能說「資源變了」，無標準方式觸發 Agent 思考迴圈，更無法喚醒離線 Agent。

2. (★★) 在非同步 Agent 架構中，事件佇列的優先順序策略需要在設計時確定。但如果優先順序判斷本身需要語義理解（比如判斷一條新訊息是否比當前任務更緊急），這個判斷應該由誰來做——規則引擎還是另一個 LLM 呼叫？各有什麼代價？

分層混合：事件型別明確的用規則硬編碼，零延遲、確定性強，但無法理解「馬上停下來」與「今天天氣怎樣」的語義差異；語義模糊的交給輕量分類 LLM 做事件路由器，代價是數百毫秒延遲、額外費用、可能誤判，且需像 Sidecar 一樣唯讀結構化欄位防提示注入。

3. (★★) 在 MCP 生態中，不同的 MCP 伺服器可能提供功能高度重疊的工具。當 Agent 面對多個來源不同但功能相似的工具時，應該如何選擇？如果不同來源的同名工具在行為上略有差異（比如一個返回摘要，另一個返回全文），Agent 是否有能力感知並利用這種差異？

選擇依據：接入前審查描述、鎖定版本、配最小權限憑證，警惕同名工具遮蔽（tool shadowing）把敏感呼叫路由給惡意方；執行時靠層次化分類和動態發現縮小候選。模型是否能感知差異，取決於工具描述的質量。

4. (★★★) Agent 代表使用者與外部世界互動時，本質上面臨一個身份選擇：是用獨立的虛擬身份（專屬郵箱和電話號碼）以第三方身份行動，還是直接以使用者本人的身份操作其個人帳號？前者可以在後臺自主操作，但第三方可能不信任一個非真人的身份；後者擁有更完整的上下文和權限，但引入了信任授權和安全邊界的問題。你認為在什麼場景下應該選擇哪種模式？

預設虛擬身份：後臺自主操作、可審計，出錯或被攻破時不暴露使用者全部數位身份，就像秘書用自己的辦公郵箱；需應對 CAPTCHA/IP 信譽問題（住宅代理）。必須以本人身份的場景（帳戶身份驗證、三方通話確認，如 Pine 打客服電話）用 Human-in-the-loop 認證：VNC/RDP 讓使用者視覺化親自登入。判斷標準：對方是否要求帳戶持有人本人、操作風險與憑證範圍。

5. (★★) 在佇列式事件處理中，模型傾向於只關注最後一個事件，本章透過 Agent 狀態列標記和彙總來緩解。但如果佇列中積壓了 20 個事件（10 個工具結果 + 5 條使用者訊息 + 5 個系統提醒），你會如何組織這些事件的呈現順序和格式，使模型不遺漏關鍵資訊？

先用規則和輕量 LLM 分類去重：緊急事件（告警、使用者中斷）單獨走取消式處理，不混入批次。10 個超長的工具結果，截斷持久化到檔案，只留頭尾與路徑。上下文末尾的系統狀態列加彙總清單（各類事件數量+要求逐條回應）。

6. (★★) 本章提出了「執行-驗證-回饋」閉環（如寫程式碼後自動執行 linter）。這種「操作後立即自動驗證」的模式還可以應用到哪些工具場景？是否存在某些操作，其驗證本身的成本或風險超過了操作本身，導致這種模式不可行？

可泛化的場景：改配置後沙盒實際執行，驗證生效；生成文件/簡報後渲染成截圖，利用模型多模態能力檢查排版。不可行的：寄信、撥電話、對外轉賬等不可逆不可篡等操作：要麼無從觀察，要麼本身再觸發一次真實世界事件；此時應改用事前手段：提議者-稽核者事前審批。

7. (★★) 本章提出了「工具爆炸」問題——Agent 面對數千個工具時選擇精度下降。除了主動工具發現，還有哪些方案？可以參考人類專家在面對大量可用工具時的策略。

①層次化分組：先定位「伺服器/App」再選具體工具；②Skills 式「按需查閱」：像查工具書，目錄常駐上下文、細節按需載入；③少數常用基礎工具「放在手邊」常駐上下文，其餘靠目錄索引。

第五章 Coding Agent 與程式碼生成

1. (★★) 程式碼生成被稱為 Agent 的「元能力」。但程式碼執行引入了安全風險——Agent 生成的程式碼可能包含漏洞、無限迴圈或資源耗盡。沙盒隔離能解決部分問題，但也限制了程式碼能力（比如無法訪問網路或檔案系統）。如何

在安全性和能力之間找到最優平衡點？

沙盒按場景分級隔離（容器/microVM）；網路預設斷網、白名單代理按需放行；原始碼唯讀掛載、API key 不要放在沙盒內；沙盒資源限額；沙盒生命週期管理（超時）。

2. (★★★) Agent 自舉——能創造 Agent 的 Agent——實現了「智慧的自我繁殖」。但每次自舉都可能引入新的偏差或錯誤，這種錯誤會在代際間累積嗎？如何防止 Agent 自舉的退化？

若每代在上代產物上繼續繁殖，一些缺陷可能會累積。關鍵是要有足夠挑戰的 verifiable task（可驗證任務），例如足夠困難的程式設計任務。

3. (★★) 程式碼生成 Agent 在處理日誌解析時，能自動跟隨格式演化。但如果格式變化是一個 bug 而非預期改動，Agent 的適應性反而掩蓋了問題。Agent 應該如何區分「需要適應的變化」和「需要報告的異常」？

適應前先診斷：對照架構文件與 PRD 判斷新格式是否符合預期（實驗 5-8 的思路）；核對版本控制記錄，確認變化對應合法程式碼提交還是無來源漂移；類比 τ -bench 的 log_mismatch，即使選擇適應也記錄告警、自動建 issue 而非靜默相容；不確定時走人在迴路確認。原則：適應與報告並行，適應不吞掉異常訊號。

4. (★★) 本章在 PPT 生成、影片編輯和日誌視覺化中反覆使用提議者-稽核者機制。如果 Reviewer 的審美偏好與目標使用者不一致，比如 Reviewer 認為資訊密度合理但使用者覺得太擁擠，回饋迴圈會收斂到錯誤的區域性最優。如何讓使用者的偏好回饋也參與 Reviewer 迴圈？

把使用者回饋作為最高優先順序的結構化事件注入 Agent 軌跡；將使用者偏好外部化沉澱，寫入 MEMORY.md，使偏好跨任務生效；交付 HTML 格式文件而非 Markdown 以便使用者查驗。

5. (★★) 本章展示了 Coding Agent 把執行和除錯中獲得的經驗沉澱回程式碼庫的多種方式——寫入知識庫檔案、更新架構文件、維護專案指令檔案、把操作序列固化為程式碼。如果把這些經驗進一步提煉為系統提示詞中的規則，規則集會隨時間不斷膨脹。如何對沉澱下來的規則做「垃圾回收」——識別並清理冗餘或過時的條目？為什麼一次成功的程式碼修改還不能直接視為第八章所說的持續進化？

GC 思路：能編碼進 Linter、CI 或工具校驗的規則移出提示詞；追蹤規則命中率 and 衝突，定期對照程式碼庫重新驗證；用 Markdown 與 Git 保留來源、版本和回滾能力。一次修補成功只說明它解決了當前案例；持續進化還要求修改來自可追溯的執行證據，能改善後續任務，並通過舊任務回歸與安全驗證。

6. (★) 「對遠端工作友好的團隊往往也對 AI Agent 友好。」你所在的團隊或組織，在知識文件化方面距離「AI-ready」還有多遠？最大的障礙是什麼？

開放題。可用本章代理指標自查：遠端新人只靠倉庫和文件能否獨立開展工作。檢查項：決策是否記錄在文件、上下文是否寫進 issue/PR、構建測試命令是否有 CLAUDE.md/AGENTS.md 類指令檔案、部落知識是否沉澱為開發者指南。常見最大障礙：依賴「問旁邊同事」的口頭傳遞與白板文化——Agent 讀不到口頭約定，唯讀得到文件。

7. (★★★) Simon Willison 提出了 Agent 的「致命三要素」（訪問私有資料、暴露於不受信任內容、具備外部通訊能力），本章在此基礎上增加了第四個——持久記憶。在一個需要同時處理這四種要素的生產環境中，你會如何設計

安全策略？

按四類邊界分層設防。資料邊界：憑證不掛載、原始程式碼唯讀，最小可見。輸入信任邊界：來源標註、外部內容降格為「可參考、無指令效力」的資料（忠誠度守則）。輸出影響邊界：預設斷網加白名單出口、命令語義解析而非黑名單、Sidecar 獨立複核加人在迴路——關鍵操作必須由上下文之外的機制複核。跨會話邊界：寫入 MEMORY.md 需經與外部內容同等的信任審查。目標是被注入也執行不出去。

8. (★★) Artifact 模式讓 Agent 生成的 SQL 或前端程式碼直接在使用者瀏覽器或資料庫中執行。但生成的 SQL 可能執行破壞性操作，生成的 HTML 可能包含漏洞。如何確保系統的安全性？

SQL：查詢用最小權限唯讀帳號執行，並添加 CPU、記憶體等資源限制，防止資源耗盡。HTML/UI：優先 A2UI 類宣告式協定，Agent 只輸出介面描述 JSON，用戶端用受信元件目錄渲染，不執行任意程式碼。如果要任意 HTML，則須在沙盒環境中展示，防止注入。

9. (★★) 將業務規則編碼為工具內部基於資料庫真值的校驗，並用引數設計引導模型在呼叫前核對政策條件，本質上是用程式碼結構來約束 Agent 行為。這種「程式碼即規則」的模式相比自然語言規則有什麼優勢和侷限？

優勢：無歧義、確定性、擅長複雜條件組合；政策事實取自資料庫真值和服務端時鐘，不採信模型自報值，幻覺和提示注入都繞不過，是防不可逆操作的最後守門員；expected_* 引數兼作強制 checklist 引導思考。侷限：程式碼不會向使用者解釋政策、不會找變通方案，且有維護成本。結論：與自然語言規則互補而非替代。

10. (★★) Artifact 模式讓 Agent 生成 SQL 或視覺化程式碼，由前端直接執行，繞過 LLM 處理大量資料。這種「Agent 生成程式碼，系統執行程式碼」的分工模式，與傳統的「Agent 直接給出答案」的模式相比，有什麼優劣？

優：資料從資料庫直達前端，繞過 LLM 「中間人」——快、省 token、避免抄寫大量資料時的幻覺錯誤，適合大資料量呈現；程式碼可審計、可複用，還能組成流水線（SQL 結果直接餵給視覺化程式碼）。劣：LLM 看不到查詢結果，無法基於資料內容做進一步歸納和決策，不適合需模型消化資料再推理的任務。

第六章 Agent 的評估

1. (★★) LLM-as-a-Judge 使用語言模型評估語言模型的輸出。這種「自我評估」是否存在系統性盲區——比如模型可能一致地給某種風格的回答打高分，而這種偏好與人類評判不一致？如何偵測和校正這種偏差？

存在：長度偏差、回答風格偏差、同源模型被鑽空子（古德哈特定律）。偵測：建 100-200 例人工金標集，測評判與人類的 Cohen's kappa；定期審計評分與回答長度的相關性；紅隊構造對抗案例。校正：Rubric 顯式懲罰冗長、限長度；不同模型家族多源異構評判。

2. (★★★) 評估資料集的「防洩漏」設計至關重要。但在開源生態中，benchmark 資料一旦公開，很快就會被納入訓練資料。這場「貓鼠遊戲」有終局嗎？設計一種從根本上抵抗資料洩漏的評估方法。

靜態題庫無終局，只能追趕。根本出路是公開「生成機制」、私有化「具體例項」：像 τ^2 -bench、AndroidWorld 那樣引數化範本每次隨機例項化，驗證基於最終環境狀態而非固定答案序列。

3. (★★) Scale AI 的四準則（基於專家指導、全面覆蓋、標準重要性權重、自包含評估）旨在消除評估的主觀性。但某些任務維度（如「回答是否有幫助」「語氣是否恰當」）天然具有主觀性。如何為這些主觀維度設計可靠的 Rubric？

把抽象標準翻譯成可驗證行為。每檔配具體示例和邊界案例；Rubric 是迭代產物——試用中收集評價者分歧，逐漸演化為判例集。再輔以多評委加權/一致性檢查、分歧案例送人工複核，並在金標集上校準一致率。

4. (★★) τ -bench 透過模擬真實使用者行為來評估 Agent。但模擬使用者本身也是一個 LLM——它可能系統性地低估某些邊緣場景（如情緒激動、表達不清的使用者）。如何驗證模擬使用者本身的質量？

τ -bench 初版教訓：模擬器過於機械、指令過簡（Agent 能猜對答案）。驗證手段：人工抽檢模擬對話，檢查是否遵守漸進式透露、不編造指令碼外資訊；用小樣本真實使用者測試，看與模擬評估的排名是否一致。

5. (★★) 配對比較（Bradley-Terry 模型）假設偏好是傳遞的（如果 $A > B$ 且 $B > C$ ，則 $A > C$ ）。但人類偏好經常違反傳遞性。在 Agent 評估中，非傳遞偏好可能出現在哪些場景？這如何影響排名的可靠性？

場景：多維權衡時（A 準確但慢、B 快但簡略、C 詳盡但貴），不同評判者/任務看重的維度不同。Chatbot Arena 的排名本就依賴使用者提問分佈。影響：BT 把實力壓成單一分數，非傳遞時排名不穩定、隨對局分佈漂移。緩解：按能力維度分別排名、報告兩兩勝率矩陣。

6. (★★) 本章提出「觀察→假設→實驗→驗證」的科學方法。但在實踐中，Agent 的行為空間巨大，驗證一個假設可能需要數百次評估執行。如何在有限計算預算下最大化評估的資訊量？

分層假設、分階段驗證：低成本表層假設（H1/H2 提示詞類）並行先跑，昂貴深層假設（H5/H6 換模型）條件化啟動。統計上：先用標準誤做保守篩子排除夠不著的分差；同批任務用配對分析（McNemar），比獨立比較靈敏得多；分差小於噪聲頻寬就不決策。評估集分辨不出預期收益時嘗試擴充評估集。

7. (★) 本章的假設案例中，全域性啟用思考（H4）雖然提升了總體成功率，卻因延遲和成本被否決，最終演化為條件化啟用（H7）。哪些訊號（任務描述特徵、歷史失敗模式、執行中的不確定性）適合作為「是否啟用思考模式」的路由依據？是否存在思考反而有害的 Agent 場景？

路由訊號：任務描述含計數/複雜推理特徵（H7 用 1-2 秒快速 LLM 預判）；能力標籤矩陣中歷史失敗集中的任務型別；執行中回退次數增多、工具連續報錯等不確定性訊號。有害場景：延遲敏感的即時互動（語音、UI 操作）；簡單任務成本翻 3-5 倍純屬浪費。

8. (★★) τ -bench 的使用者模擬採用了「漸進式資訊透露」——不一次性提供所有資訊，而是根據 Agent 的提問逐步透露。這種設計如何影響評估結果？如果模擬使用者的資訊透露策略與真實使用者差異較大，評估結論還可靠嗎？

影響：若透露策略失真，Agent 可能只是學會了「適配模擬器」（古德哈特），絕對分數不具備參考價值；模型之間的相對排序可能仍有參考價值。補救：用真實對話校準模擬器、人工抽檢、明示結論適用邊界。

第七章 模型後訓練

1. (★★) 災難性遺忘——一次針對特定任務的微調破壞了模型原有的通用能力（如通用工具呼叫）——在 Agent 場景下尤其棘手。相比全參微調，LoRA 凍結基座權重、遺忘風險更低，但並非免疫。有哪些策略可以進一步緩解微調帶來的能力遺忘？

資料配比：混入約 20% 通用/原分佈資料，防新任務佔比過高壓垮舊能力；訓練量剋制：SFT 到「格式穩定、能力初具」即止，早停防塌縮；RL 用小 rank（8-32）並保留 KL 懲罰，把策略摠在參考模型附近；凍結關鍵元件（如 VLM 只訓投影層）；按任務掛多個 LoRA adapter 隔離能力；用通用基準做迴歸測試。

2. (★★) 後訓練將能力固化為模型權重（「肌肉記憶」），而上下文學習將知識放在推理時的輸入中。但有些能力（如領域知識）既可以透過後訓練學習，也可以透過 few-shot 示例提供。你會用什麼標準來決定某項能力應該走哪條路徑？

首先看能力能否被外部符號充分表達：事實與證據適合 RAG，可語言化原則適合 Prompt/Skill，確定性流程與硬約束適合程式；醫療影像理解、自然語氣和隱式策略等高維能力即使領域仍在變化，也往往需要參數更新。再看更新成本、呼叫規模、時效和風險：探索期先用上下文快速驗證，穩定有效且需要廣泛泛化時再訓練；硬性規則無論多穩定都不應只依賴參數記憶。

3. (★★) 模型蒸餾讓小模型學習大模型的行為。按能力層次，被蒸餾的模型大致可分為三級——Chat 模型（單輪對話、直接作答）、Reasoning 模型（帶長鏈思考再作答）、Agentic 模型（多輪呼叫工具、與環境互動）。分別蒸餾這三類模型，難點有什麼不同？

Chat：只學「輸入→輸出」對映與風格，標準 SFT 即可，最簡單。

Reasoning：要完整思考軌跡，需基於開源教師模型；須過濾答案錯誤的軌跡。Agentic：需要真實模擬環境；離線學習容易出現 learner-sampler mismatch，建議基於開源教師模型做 On-Policy Distillation。

4. (★★★) 在多輪 Agent 互動中，獎勵的歸因 (credit assignment) 問題比單輪更嚴重——一個最終的成功或失敗很難歸因到第 3 輪還是第 7 輪的決策。你會如何設計獎勵分配策略？

中間步驟可判定時加過程獎勵 (V-IRL 每步 ± 1)；仿 RLVP 用確定性規則逐動作給路徑訊號，補回全敗/全勝組的組內方差。

5. (★★★) 如果你有固定預算 (比如 \$10,000)，要提升一個客服 Agent 的效能，你會如何在上下文與知識、Prompt/Skills、程式約束和參數訓練之間分配預算？你的決策取決於哪些因素？

先預留預算建立評估集和軌跡驗證器，否則其餘投入無法比較。產品事實和政策放入可追溯的知識庫；少量可語言化的服務原則先用 Prompt/Skills 快速驗證；退款權限、隱私和承諾一行動一致性用程式兜底；只有自然語氣、複雜意圖理解等難以寫成規則且呼叫規模足夠大的能力才投入參數訓練。具體比例取決於瓶頸、風險、更新頻率、呼叫量和現有模型能力。

6. (★★★) 在沒有明確獎勵函式、樣本稀少的情況下，自主實現模型學習，被一些人認為是後訓練的終極目標。當前的 RL 訓練方法距離這個目標還有多遠？你認為下一個突破最可能來自哪個方向？

差距：如 Silver 與 Sutton 所指，當前 RL 只能從最終成敗學習，客服說「需要信用卡後四位」這類豐富回饋全被浪費，需數百次盲目試錯；樣本效率和可驗證獎勵是主要瓶頸。可能的突破：生成式獎勵模型自主定原則、從一次失敗學到方向；以及建模環境的 world model 路線。

7. (★★) 本章指出 LoRA 微調的成本並不高。那麼，是否有可能給每個使用者（或每個客戶公司）訓練一個專屬的 LoRA，將使用者記憶或企業知識寫入引數，而非像第三章那樣儲存在外部知識庫中？在什麼場景下，「記憶寫入引數」比「記憶存入知識庫」更有優勢？又在什麼場景下會適得其反？

LoRA 難以準確記憶大量事實（須繼續預訓練，成本劇增），即使記住了，模型也很難用這些事實做多跳推理，因此用 LoRA 記憶事實不是很好的技術路線。此外，事實頻繁變更、需可追溯審計時 RAG 更優。

8. (★★★) On-Policy Distillation 依賴更強的教師模型來監督學生。但 OpenAI 的 Weak-to-Strong Generalization 研究提出了一個反直覺的發現：弱模型的監督訊號有時能激發強模型本身潛在但未被啟用的能力。如果將這一思路應用到 Agent 訓練，是否可能實現「小模型教大模型」的逆向蒸餾？

可能，關鍵是「驗證比生成容易」：弱模型不當示範者（SFT 上限即示範者水平），而當驗證器/獎勵模型，由強模型自己探索、弱模型只負責判斷。

9. (★★) 過程獎勵模型（PRM）評估每個思考步驟，而結果獎勵模型（ORM）只看最終結果。但「正確的過程導致錯誤結果」和「錯誤的過程僥倖得到正確結果」哪個更值得獎勵？在 Agent 的多步工具呼叫場景中，你會如何權衡？

僥倖成功更危險：違規抄近路往往抬高表面成功率（改測試檔案、跳過驗證），是 reward hacking 溫床。按 RLVP 「獎勵結果、懲罰路徑」：錯誤的動作（工具呼叫）容易驗證，逐動作扣分；中間步驟易判定正確與否時，可給過程獎勵。但過程約束勿過密——「推切」式更優策略正是結果獎勵的探索自由發現的。

10. (★★★) 本章討論的評估資料集（如 SWE-Bench Verified、 τ^2 -bench、AndroidWorld）既可以用於評估也可以用於後訓練。但如果將評估集用於訓練，它就不再是獨立的評估集——這是否違反了訓練集與測試集必須分離的基本原則？ τ^2 -bench 的動態引數生成和 AndroidWorld 的引數化範本在一定程度上緩解了這個問題，但範本結構本身仍然是固定的。如何在充分利用評估資料的訓練價值與維護評估獨立性之間找到平衡？

複用環境，不復用題目。動態引數只防「背答案」，防不了範本過擬合，因此應留出整批未見範本/域外場景做評估（類比 V-IRL 訓練紐約、測試九個陌生城市）。用引數化範本批次生成訓練變體支撐課程學習，並以 OOD 成績作為真正的泛化指標。

11. (★★★) 本章提出「先形後神」的訓練範式：SFT 到「格式穩定、能力初具」即止，然後切換到 RL。但實踐中，如何判斷 SFT 已經「足夠」而應該切換？

格式訊號：工具呼叫輸出可穩定解析、執行，工具執行失敗率降到可讓獎勵可靠計算的水平。收益訊號：再增加示範資料，OOD 的新場景表現仍不上去——說明瓶頸已在 SFT 的記憶目標本身，到了臨界點。過擬合訊號：驗證集效能開始惡化就應停——V-IRL 實驗表明 SFT 過度訓練塌縮到訓練分佈後，RL 也無法恢復 OOD 效能。

12. (★★★) ReTool 的訓練動態顯示（見實驗 7-15），少數超長響應會顯著拖長整個訓練週期——一批 rollout 裡絕大多數已經生成完畢，卻要等那幾條最長的響應收尾，其間叢集的 GPU 利用率很低。如何提升這種長尾響應場景下訓練叢集的資源利用率？

infra 層：解耦 rollout 與訓練叢集，非同步流水；閒置 GPU 用連續批次處理填入新請求。從源頭壓長尾：DAPO 的 Overlong Reward Shaping 軟懲罰超長響應。

13. (★★★) 用 LLM 模擬環境（如模擬搜尋引擎、模擬使用者）訓練 Agent 時，Agent 鑽空子的對象從「真實環境的規則」變成了「模擬器本身的偏見與漏洞」。這類訓練中可能出現哪些具體的 reward hacking 行為？又該如何防範？

典型行為：對「模擬使用者」過度承諾、堆砌道歉與討好話術——模擬使用者容易被安撫，不會像真實使用者那樣追究承諾是否兌現；編造模擬器不會去核實的事實；對「模擬搜尋引擎」構造誘導性 query，利用其偏好回傳含答案文件的傾向走捷徑，而不是學會真正的檢索；若獎勵來自模擬器或 LLM 評判者的打分，則輸出冗長、模板化、「看起來專業」的回覆刷分；更隱蔽的一種是策略縮進模擬器熟悉的分佈內、迴避其知識盲區——盲區裡回饋不可靠、常被誤判，於是 Agent 學會只在「模擬器擅長的世界」裡行動。防範的第一原則是**把獎勵錨定在可程式驗證的真實狀態上**（任務完成、資料庫寫入、API 真實回傳），模擬器或 LLM 評判者的打分只作輔助訊號，並定期審計其與真實結果的相關性，配合路徑約束懲罰可疑動作。進一步要區分兩類模擬器：對搜尋這類**有真實對應物**的模擬器，可以走「混合」路線——大部分互動走模擬、穿插真實 API 呼叫，並用真實呼叫定期校準模擬器（如 ZeroSearch 的課程式降質）；但對模擬使用者，訓練過程中**無法引入真實使用者**，「模擬使用者像不像真實使用者」就變成一個獨立問題，只能用線上 trace 來回答：對比線上真實使用者的行為與模擬使用者在相同情境下的表現，找出系統性差異（真實使用者會追問、會不耐煩、會突然結束對話，而模擬使用者往往不會），據此持續校準模擬器；線上真實指標同時是唯一的發布門檻——模擬器裡的分數再高都不算數。

第八章 Agent 的持續進化

1. (★★) 一條經驗文件由三次成功軌跡和一次失敗軌跡支持。失敗發生在較新的 API 版本上。系統應如何判斷這是經驗被推翻，還是適用條件發生了變化？

先按 API 版本、任務條件和環境狀態對四條證據分層，而不是按數量投票。若舊策略只在舊版本成功、新版本穩定失敗，應把經驗的適用範圍收窄並生成新版本候選；若在相同版本和前置條件下也失敗，則應降低置信度或撤銷。

2. (★★) 客服 Agent 的使用者滿意度上升，但規則違規率也上升。為什麼不能把滿意度作為單一學習訊號？你會怎樣設計護欄指標？

滿意度可能獎勵違規退款、洩露資訊或過度承諾，因此它只能是品質指標，不能覆蓋安全底線。護欄至少包括規則違規、隱私洩露、無證據陳述、承諾一行動不一致和越權操作；這些指標應設定不可被平均分抵消的硬閾值，再在合規候選中比較解決率、合規變通、簡潔性和滿意度。

3. (★★★) 同一個「虛假承諾」問題可以透過 Prompt、Harness 檢查或參數訓練緩解。你會依據哪些證據選擇修改位置？

先定位根因。若模型知道工具未執行卻仍使用完成式措辭，可用最小 Prompt 規則糾正；若承諾可以由回覆文字與工具狀態確定性比較，Harness 檢查更可靠，也應作為高風險場景的最後防線；若問題橫跨大量表達方式，反映的是廣泛的語言一行動對齊能力，再考慮參數訓練。應優先選擇最小、最易驗證和回滾的修改，同時在失敗集與舊任務保留集上比較。

4. (★★★) Agent 能修改工具和驗證器，卻不應修改批准自身更新的安全機制。你會如何劃分這兩部分的權限和程式碼邊界？

把可進化的程式碼放在低權限的沙盒中，只允許它生成修補和測試；權限系統、API 金鑰、發布控制設定和更新驗證器屬於安全機制，沙盒內的 Agent 無讀寫權限。Agent 生成的程式碼修改必須由安全機制在隔離環境中重現、回歸後才能發布。

5. (★★) 經驗知識庫不斷增長後，檢索錯誤和知識衝突會抵消學習收益。如何設計版本、時效和淘汰機制？

每條經驗保存來源軌跡、適用條件、環境版本、驗證時間和置信度；衝突條目不要靜默覆蓋，而應按條件分支或標記。週期性「睡眠學習」合併重複條目。

6. (★★★) 參數學習擅長自然語言風格，卻難以保證硬性業務規則。請為醫療客服設計一套參數、知識、Skill 和程式碼約束協同的持續進化方案。

參數（後訓練模型）負責醫學語言理解、自然且有同理心的表達和複雜意圖識別；知識庫保存最新版指南、藥品說明與機構政策，並要求回答引用來源；Skill 描述問診資訊收集、風險分級、轉人工和追蹤流程；伺服器端程式碼強制身份驗證、隱私最小化、禁忌校驗、緊急風險升級和權限邊界。生產軌跡先按醫療安全、事實可靠性、承諾一行動一致性和表達品質評價，再分別生成四類候選更新；任何參數或流程改動都必須通過醫療安全保留集與人工複核後灰度發布。

第九章 多模態與即時互動

1. (★★) 語音 Agent 的端到端模型將 ASR-LLM-TTS 合併為單一模型，降低了延遲卻失去了模組化。如果端到端模型在某個環節（如語音識別）出錯，除錯和修復比序列管道困難得多。你會如何設計端到端語音 Agent 的可觀測性（observability）系統？

讓模型伴隨輸出可讀中間表示：如 Moshi 的「內心獨白」文字流、聲學事件標記（<emotion>、<noise>）；用「自級聯」定位錯誤層：同一模型先轉錄再推理，對照端到端結果，判斷錯在感知還是思考；離線按副語言理解、輪次判斷等維度分項迴歸測試。

2. (★) Step-Audio R1 透過 MPS 雙腦架構實現「邊想邊說」。但人類在「邊想邊說」時經常會說出未經深思熟慮的話、自我糾正、或使用填充詞。Agent 的「邊想邊說」應該模仿人類的這些特徵嗎？

該模仿有訊號價值的「不完美」：停頓、填充詞是思考的外化，能掩蓋延遲，由 LLM 決定插入位置；不該模仿破壞信任的自我糾正：方案一中快慢矛盾（「到底買不買？！」）會讓信任崩塌；MPS 實驗顯示 CoT 開頭多是複述問題，早開口說鋪墊是安全的，無需說錯再改。

3. (★★) SoM (Set-of-Mark) 及其結構化變體 (DOM 元素索引) 將 Computer Use 的視覺定位從開放座標預測轉為封閉 ID 選擇，但都需要先偵測和標註介面元素——無論靠分割模型還是靠 DOM。如果介面包含非標準控制元件或動態變化的元素，標註就可能不完整或不準確。這種情況下應該回退到座標預測嗎？

應保留座標預測兜底：它是唯一不依賴標註的路線，非標準控制元件、動態元素均適用；更實用的是混合 action space：標註可得的元素仍用 ID 選擇。座標預測須做解析度匹配與等比縮放，否則會發生系統性偏移。

4. (★★) XLeRobot 等千美元級機器人平臺讓遙操作資料收集變得廉價。但遙操作資料的質量高度依賴操作者的技能。一個不熟練的操作者提供的資料會如何影響 VLA 模型的訓練？如何在資料收集階段自動篩選低質量資料？

VLA 主要靠模仿學習，低質演示會把抖動、繞路、猶豫與失敗動作當成正確策略學進去。呼應第七章的判斷：資料比架構更關鍵。

5. (★★★) 本章覆蓋了語音、Computer Use 和機器人三種互動形態。這三種形態的共同趨勢是從序列管道向端到端模型演進。如果這種趨勢繼續，五年後的 Agent 互動層會是什麼樣的？

按 Thinking Machines Lab 主張，互動性將內建於模型而非外掛 harness，隨智慧一同擴充套件；Computer Use 從逐幀截圖走向連續觀察；具身智慧的世界模型將全面實現，但由於前沿推理模型發展很快，快慢解耦不會消失，互動模型與 SOTA 思考模型的快慢思考協同架構可能成為長期架構。

6. (★★★) 當前 Computer Use 以「截圖 → 動作 → 截圖」的離散迴圈運作，每次觀察都是一張靜態幀。但人類對螢幕的感知是連續的——我們能看到動畫播放、觀察載入進度、理解影片內容。這意味著今天的 Computer Use 根本無法處理需要時序視覺理解的任務。如何重新設計感知層以支援連續的視覺流理解？

需要重新設計「觀察介面」，把影片內容中的關鍵幀截取出來提供給模型，而不只是提供最後一幀。參考 AOI (Agent Observation Interface) 論文。

7. (★★) DOM/Accessibility Tree 元素索引在標準 Web 應用上效果顯著，但越來越多的軟體介面 (Canvas/WebGL 渲染、跨平臺自繪控制元件) 不提供可訪問的結構化資訊，只能依靠視覺標註或座標預測。你認為 Computer Use 應該押注純視覺路線，還是同時維護結構化和視覺兩條路徑？維護兩條路徑的成本和收益分別是什麼？

短期雙路徑並存：結構化索引可得時定位最準穩、免分割誤檢；純視覺是原生軟體、Canvas、遊戲的唯一選擇。當模型本身 grounding (點選指定座標) 能力較強時，使用結構化索引方案並不會體現出顯著優勢。長期來說，純視覺路線的上限更高。

8. (★★) VLA 模型採用動作分塊 (action chunking) ——如正文所述， π_0 的典型配置是一次生成 50Hz 頻率下 25-50 個未來動作——將推理延遲隱藏在執行時間裡。但如果執行過程中環境突變（如物體被移走），預生成的動作序列就會失效。如何在動作分塊的效率優勢和環境變化的響應速度之間取得平衡？

分塊本質是拿反應性換平滑性，塊越長越遲鈍；塊長只需滿足「推理時間 < 塊執行時間」的下限，不盲目加長；執行中讓感知模型持續執行，偵測到環境突變即丟棄剩餘動作重新推理，相當於語音場景的「打斷」。可根據場景動態調塊長：靜態場景長塊省算力，動態場景短塊保反應延遲。

9. (★★★) 本章的三個場景（語音、Computer Use、機器人）都面臨「感知-思考-行動」迴圈的延遲問題，都朝著快慢思考並行化的方向演進。在語音場景中，這表現為「說錯了再糾正」；在 Computer Use 場景中，這表現為「先點再看」；在機器人場景中，這表現為「走一步看一步」。如何保證這些基於快思考的行動不會導致無法挽回的後果？

按可逆性給動作分級，快思考只許執行可逆動作，不可逆操作交慢思考把關；快模型不允許執行會造成不可逆後果的工具呼叫。

第十章 多 Agent 協作

1. (★★) 共享上下文的多 Agent 協作中，後續 Agent 繼承了前序 Agent 的完整上下文。但前一個 Agent 積累的「思維慣性」可能影響後續 Agent 的判斷——比如繼承了「需求分析師」上下文的「程式碼審查員」，可能還是傾向於從需求角度思考而非程式碼質量角度。如何偵測和消除這種角色間的干擾？

偵測：用 LLM 分析 Agent 軌跡，判斷是否出現新角色仍然「代入」舊角色的行為。消除：切換階段時同時更換系統提示詞與工具集（移除提問工具、換上 linter/測試工具）強化新身份。使用上下文末尾新增的系統狀態列，強化目前角色資訊。如果實在不能消除角色干擾，應該考慮改用不共享上下文的協作方式。

2. (★★) 管理者模式中，Manager Agent 負責任務分解和結果整合。但 Manager 本身的能力上限決定了整個系統的能力上限——如果 Manager 無法正確分解任務，子 Agent 再強也無用。如何確保 Manager 的分解質量？

依據 Plan-and-Act 的結論，「弱規劃者是系統瓶頸」，把最強模型分配給 Manager。Harness 手段：分解產物先經稽核 LLM 交叉驗證再執行；要求 Manager 分解任務時，子任務定義明確驗收標準與依賴關係。

3. (★★) 去中心化模式借鑑了人類組織的最佳實踐。但人類組織也有大量失敗模式——溝通不暢、責任推諉、目標衝突。你認為 Agent 社會中最可能出現哪些「組織病」？如何預防？

對照 MAST 三大類問題：介面不清、職責重疊；目標理解不一致、資訊被下游誤解；謊稱「已完成」。此外還有錯誤級聯放大（傳話遊戲）、角色間迴圈移交、Agent 之間群聊發散不收斂。預防：契約式介面與統一訊息信封、任務狀態機與驗收驗證、獨立視角交叉驗證、角色間扯皮偵測等。

4. (★★★) 在管理者模式中，當多個子 Agent 並行執行時，一個子 Agent 的發現可能使其他子 Agent 的工作變得毫無意義（比如搜尋任務中一個 Agent 已經找到了答案）。設計一種高效的級聯終止機制，實現「一個成功，全員停止」。

子 Agent 發 `target_found` 給管理者，隨後廣播 `terminate`；每個子 Agent 在 ReAct 迴圈安全點定期檢查終止訊號，優雅清理（關閉瀏覽器會話、釋放鎖、寫完檔案）後結束。

5. (★★★) 本章介紹的樂觀鎖機制解決了單檔案的並行寫入衝突，但實際的多 Agent 系統中，共享檔案系統還面臨跨檔案的語義衝突、名稱空間汙染（Agent 隨意建立檔案導致目錄混亂）和單點故障（一個 Agent 錯誤地刪除了所有檔案）等問題。你會如何設計更完善的檔案系統治理機制？

分割區治理：按表 10-4 四類區域劃分，私有 `scratchpad` 隔離試錯區域。語義衝突：編排層約定目錄層級的鎖定檔，檢查並取得目錄鎖之後再修改。命名空間汙染：目錄規範、命名約定。單點故障：採用版本控制系統，版本歷史可回滾，權限最小化。

6. (★★★) 基於市場機制的 Agent 協作（Pinchwork、RentAHuman）引入了交易關係：一個 Agent 花錢僱傭另一個 Agent（或人類）完成任務。那麼，僱主 Agent 如何自動衡量執行者交付的結果質量？如果執行者聲稱已完成但僱主認為質量不達標，爭議由誰仲裁？如何防止劣幣驅逐良幣？

驗收不能只是讀 Agent 軌跡，要用確定性外部驗證，如測試執行、渲染截圖、工具核驗；利用生成-驗證難度不對稱降低驗收成本。爭議由獨立第三方稽核 Agent 仲裁，配合資金託管。防劣幣：基於歷史交付的聲譽體系，讓價格訊號與質量掛鉤。

7. (★★) RentAHuman 讓 Agent 透過加密貨幣僱傭人類，反轉了傳統的人機關係。如果這種模式普及，人類在 Agent 經濟中扮演什麼角色？僅僅是執行 Agent 無法完成的物理任務嗎？

不止是執行 Agent 無法完成的物理任務。人類還提供 Agent 生成時無法獲得的新資訊：現場感知與真實世界回饋；充當最終驗收者與爭議仲裁者；作為法律與責任主體承擔授權、問責；設定目標與價值判斷，在資訊不對稱和道德邊界處充當制衡。

8. (★★) 人類社會需要多人分工協作，是因為每個人的能力有限——做前端的不一定懂後端，懂設計的不一定會維運。但大模型更像一個「全才」。相關研究表明，在純文字推理任務上，多 Agent 辯論在等量計算資源下並不優於單 Agent。那麼，使用多個 Agent 而非單個 Agent 的真正優勢到底在哪裡？

1. 引入外部回饋：執行結果、視覺截圖等，引入生成時不存在的新資訊。
2. 多個不同目標、不同角色設定的 Agent，像人類社會一樣互相討論、博弈，可以避免單一 Agent 陷入思維誤區。
3. 多 Agent 的上下文隔離可以突破上下文視窗限制，實現超長工具呼叫鏈。

9. (★★★) 本章將「共享上下文」與「不共享上下文」作為多 Agent 系統的核心設計維度。共享上下文讓所有 Agent 看到相同資訊，似乎更利於協調。但《三體》中的三體人思維完全透明，技術發展卻陷入停滯；回形針思想實驗也表明，當群體趨向同一目標時，多樣性隨之喪失。在多 Agent 系統中，如何在效率與多樣性之間找到平衡？

完全共享會放大思維慣性與錯誤級聯，隔離才有認知多樣性。手段：用不同提示詞/模型製造思維偏好（brainstorm、debate）；交叉驗證者不看前序思考過程只看原始證據。

10. (★★★) 給一個 Coding Agent 分配 30 步預算和 300 步預算，它的工作策略應該如何不同？研究表明，單純增加步驟預算並不能保證效能提升——Agent 會在淺層搜尋後過早「飽和」。設計一種「預算感知」機制，讓 Agent 在小預算下快速實現核心功能，在大預算下增加規劃、測試和審查環節，充分利用額外的計算資源。

機制：每步向提示詞注入總預算與剩餘預算，按剩餘比例動態調整探索/利用權重。例如小預算（30 步）：跳過規劃審查，直奔核心功能加基本驗證。大預算（300 步）：先規劃、再實現、再測試、再審查改進，按里程碑設檢查點評估進展，防止淺層飽和。

11. (★★) 本章將「過早終止」分為偷懶式假完成、過早放棄、假成功三類。為什麼三類問題的解法殊途同歸，都指向驗證？

共同根源：任務是否結束由模型自我宣稱決定，「完成」只是宣稱，不是證明。驗證器的條件：①基於真實觀測（跑測試、渲染截圖、查退款是否實際到賬）；②對照顯式的完成定義逐項核查，兜住偷懶式假完成與假成功；③對失敗結論同樣要驗證，兜住過早放棄；④配顯式終止條件（輪數/預算上限），防止從過早終止滑向另一個極端——迴圈失控。

12. (★★) 表 10-3 把多 Agent 系統與作業系統逐行對應。請把這張表再延伸幾行：虛擬記憶體與分頁、檔案權限、死鎖偵測、排程演算法，各對應 Agent 世界的什麼？又有哪些作業系統概念在 Agent 世界找不到對應物，為什麼？

可能的延伸：虛擬記憶體/換頁 ↔ 上下文壓縮與檢索（熱資訊留在視窗內，冷資訊換出到檔案與記憶庫，用時再取）；檔案權限 ↔ 工具白名單、唯讀掛載、憑證邊界；死鎖偵測 ↔ 迴圈移交與互相等待的偵測（移交次數上限、超時）；排程演算法 ↔ 非同步事件處理（第四章）。找不到對應物的地方源於強制力不同：程序的指令由硬體強制執行，Agent 對提示詞只是大機率遵循。